

Chapter 66: Real-time Apps

Scenario 1: A trading dashboard that ticks every second is freezing the tab

Situation: A market-data dashboard receives price ticks over a WebSocket at roughly 200 messages/second across 150 visible symbol rows. Every tick calls a signal `.set()`, and the whole page (including a side panel with charts) becomes sluggish within seconds, with input on the search box lagging visibly.

Question: How would you redesign the update pipeline so the UI stays responsive under this message rate, and what Angular-specific mechanisms would you use?

Answer: The core problem is not "signals are slow," it's that every single tick is triggering a full synchronous render cycle across the whole component tree, and unrelated UI (the search box) is being re-evaluated because it shares a change-detection zone with the ticking rows. The fix has three layers:

1. **Isolate the hot path with per-row signals, not one big object.** If all 150 rows live in one `signal<Row[]>`, updating one row forces every consumer of that signal to re-diff all 150 rows. Instead, keep a `Map<string, writableSignal<Row>>` so each row component reads its own signal and only that row's template re-renders.
2. **Coalesce ticks per animation frame instead of per message.** Don't call `.set()` synchronously for every WebSocket message — buffer incoming ticks in a plain JS `Map` and flush them once per `requestAnimationFrame`, which naturally caps updates at ~60/sec regardless of feed rate.
3. **Use `OnPush` (default in standalone signal components) and avoid `zone.js` entirely** — zoneless change detection means only components whose signals actually changed get scheduled, so the search box never re-renders from tick traffic.

```
import { Injectable, signal, writableSignal } from '@angular/core';

interface Tick { symbol: string; price: number; ts: number; }

@Injectable({ providedIn: 'root' })
export class TickerStore {
  private rows = new Map<string, writableSignal<Tick>>();
  private pending = new Map<string, Tick>(); // coalescing buffer
  private framesScheduled = false;

  getRowSignal(symbol: string): writableSignal<Tick> {
    if (!this.rows.has(symbol)) {
      this.rows.set(symbol, signal({ symbol, price: 0, ts: 0 }));
    }
    return this.rows.get(symbol)!;
  }

  // Called synchronously from the websocket 'message' handler – cheap, no CD trigger.
  onwireTick(tick: Tick): void {
    this.pending.set(tick.symbol, tick); // last-write-wins per symbol this frame
    if (!this.framesScheduled) {
      this.framesScheduled = true;
    }
  }
}
```

```

    requestAnimationFrame(() => this.flush());
  }
}

private flush(): void {
  this.framesScheduled = false;
  for (const [symbol, tick] of this.pending) {
    this.getRowSignal(symbol).set(tick); // only ~150 signal writes, once per frame
  }
  this.pending.clear();
}
}

```

Each row component then does `rowSignal = this.store.getRowSignal(this.symbol)` and binds directly in the template. Because each row is its own signal, changing one symbol's price never touches the other 149 rows' render output, and because flushing is capped at animation-frame cadence, the browser is never asked to paint faster than it can. Tradeoff: symbols that tick multiple times within one frame silently drop the intermediate values (last-write-wins) — acceptable for a live price display, not acceptable if every intermediate tick matters (e.g., an audit trail), in which case you'd buffer an array per symbol instead of overwriting.

Interviewer intent: Tests whether the candidate understands that "signals are fast" doesn't remove the need for explicit throttling/coalescing design, and whether they know granular per-item signals beat one coarse array signal for high-frequency partial updates.

Scenario 2: Out-of-order updates are showing stale data as if it were fresh

Situation: A delivery-tracking screen subscribes to a real-time feed of driver location updates. Because messages travel over different network paths (some via WebSocket, some via a fallback long-poll), an older location update occasionally arrives after a newer one, causing the driver's pin to jump backward on the map for a moment.

Question: How do you guarantee the UI never regresses to stale state even when the transport delivers messages out of order?

Answer: Never trust arrival order — trust a monotonically increasing value in the payload itself (server timestamp or sequence number per entity). Each entity's store should reject any incoming update whose sequence/timestamp is not newer than what it already has. This is a per-key "last-writer-wins by logical clock" pattern, and it should sit in the data layer, not be sprinkled through components.

```

import { Injectable, signal, computed } from '@angular/core';

interface DriverLocation {
  driverId: string;
  lat: number;
  lng: number;
  serverTs: number; // authoritative ordering key, NOT client receive time
}

@Injectable({ providedIn: 'root' })

```

```

export class DriverLocationStore {
  private latest = signal<Map<string, DriverLocation>>(new Map());
  private lastSeenTs = new Map<string, number>();

  readonly locations = computed(() => this.latest());

  applyUpdate(update: DriverLocation): void {
    const lastTs = this.lastSeenTs.get(update.driverId) ?? -Infinity;
    if (update.serverTs <= lastTs) {
      return; // stale message - arrived late, discard silently
    }
    this.lastSeenTs.set(update.driverId, update.serverTs);
    this.latest.update(map => {
      const next = new Map(map);
      next.set(update.driverId, update);
      return next;
    });
  }
}

```

Key reasoning points to raise in an interview: (1) client-side `Date.now()` at receive time is useless for ordering because it reflects network jitter, not event order — the sequence must be assigned at the source. (2) This pattern generalizes to any per-entity real-time stream: chat message edits, collaborative cursors, sensor readings. (3) If the transport can also duplicate messages, the same guard (`<=` rather than `<`) makes the store idempotent for free. (4) A subtlety: if a client reconnects after being offline, its "resync" snapshot fetched via REST must also carry a `serverTs` /version so the store can correctly decide whether streamed updates that arrive concurrently with the snapshot are newer or older than it.

Interviewer intent: Checks whether the candidate reaches for logical ordering (sequence numbers/server timestamps) instead of naively trusting message arrival order or client-side wall-clock time.

Scenario 3: Optimistic UI update needs to roll back cleanly on server rejection

Situation: In a real-time kanban board, dragging a card to a new column updates the UI instantly (optimistic), then a WebSocket round-trip confirms or rejects the move (rejected if another user already moved it, or if the user lacks permission). Currently a rejection just logs an error to the console and the card silently stays in the wrong place forever.

Question: Design an optimistic-update mechanism that reliably reverts to the correct state on rejection, including edge cases where more updates land on top of the optimistic one before the rejection arrives.

Answer: The safe pattern is to never mutate the "source of truth" signal directly for an optimistic change — instead keep the confirmed state and a list of in-flight optimistic patches, and derive the displayed state as confirmed + pending patches via a `computed()`. On success, drop the patch (server's next confirmed event will already reflect it). On rejection, drop the patch and the computed view naturally snaps back — it never "restores a snapshot," it just stops applying a patch that was never true.

```

import { Injectable, signal, computed } from '@angular/core';

```

```

interface Card { id: string; columnId: string; title: string; }
interface Patch { id: string; cardId: string; columnId: string; }

@Injectable({ providedIn: 'root' })
export class BoardStore {
  private confirmed = signal<Map<string, Card>>(new Map());
  private pendingPatches = signal<Patch[]>([]);

  // Displayed state = confirmed truth with any still-pending optimistic patches layered on
  // top.
  readonly cards = computed(() => {
    const base = new Map(this.confirmed());
    for (const patch of this.pendingPatches()) {
      const card = base.get(patch.cardId);
      if (card) base.set(patch.cardId, { ...card, columnId: patch.columnId });
    }
    return base;
  });

  moveCardOptimistically(cardId: string, columnId: string): string {
    const patchId = crypto.randomUUID();
    this.pendingPatches.update(list => [...list, { id: patchId, cardId, columnId }]);
    this.socket.send({ type: 'move', patchId, cardId, columnId });
    return patchId;
  }

  // Server ack – success just discards the patch; the next 'card-updated' event supplies
  // real truth.
  onServerAck(patchId: string, accepted: boolean, correctedCard?: Card): void {
    this.pendingPatches.update(list => list.filter(p => p.id !== patchId));
    if (!accepted && correctedCard) {
      this.confirmed.update(map => {
        const next = new Map(map);
        next.set(correctedCard.id, correctedCard);
        return next;
      });
    }
  }

  // Real-time updates from OTHER users always land on `confirmed`, never touching
  // pendingPatches.
  onRemoteCardUpdate(card: Card): void {
    this.confirmed.update(map => {
      const next = new Map(map);
      next.set(card.id, card);
      return next;
    });
  }
}

```

Why this beats "snapshot and restore": if the user drags card A, then before the ack arrives also drags card B, a snapshot/restore approach risks reverting B's move too when A is rejected (if you naively restored a single saved previous-state blob). With independent patches keyed by `patchId`, rejecting A's patch only removes A's patch; B's patch is untouched and still displayed, and it will resolve independently. The `computed()` re-derivation means there's no manual "undo" logic to get wrong — correctness falls out of always recomputing display state from confirmed + active patches.

Interviewer intent: Tests whether the candidate can design optimistic UI that survives concurrent, overlapping optimistic operations rather than a fragile single-snapshot rollback that breaks under real-world interleaving.

Scenario 4: Reconnection after a dropped WebSocket is hammering the server

Situation: On flaky mobile networks, a chat app's WebSocket drops frequently. The naive reconnect logic (`onclose => reconnect immediately`) causes a thundering-herd retry storm when the backend restarts, and users report rapid connect/disconnect toggling that spams system notifications ("You're back online" / "Connection lost").

Question: Design a robust reconnection strategy with backoff, jitter, and UI-state debouncing.

Answer: Three separate concerns need separate handling: (1) exponential backoff with jitter so the client doesn't hammer a recovering server, (2) a max-retry ceiling with a "give up, show manual retry" state, (3) debouncing the *displayed* connection status so brief blips (reconnect within, say, 2 seconds) never surface a toast to the user — only sustained disconnection should.

```
import { Injectable, signal, computed } from '@angular/core';

export type ConnState = 'connected' | 'connecting' | 'disconnected';

@Injectable({ providedIn: 'root' })
export class RealtimeConnection {
  private rawState = signal<ConnState>('connecting');
  private displayTimer: ReturnType<typeof setTimeout> | null = null;
  readonly displayedState = signal<ConnState>('connecting');

  private attempt = 0;
  private readonly maxDelayMs = 30_000;
  private readonly baseDelayMs = 500;
  private ws?: WebSocket;

  connect(url: string): void {
    this.ws = new WebSocket(url);
    this.setRaw('connecting');

    this.ws.onopen = () => {
      this.attempt = 0; // reset backoff once a connection actually succeeds
      this.setRaw('connected');
    };

    this.ws.onclose = () => {
```

```

    this.setRaw('disconnected');
    this.scheduleReconnect(url);
  };

  this.ws.onerror = () => this.ws?.close();
}

private scheduleReconnect(url: string): void {
  this.attempt++;
  const exp = Math.min(this.maxDelayMs, this.baseDelayMs * 2 ** this.attempt);
  const jitter = Math.random() * exp * 0.3; // +/-30% jitter avoids synchronized retry
  storms
  setTimeout(() => this.connect(url), exp + jitter);
}

// Debounce what the UI shows: a blip that recovers within 2s never becomes visible churn.
private setRaw(state: ConnState): void {
  this.rawState.set(state);
  if (this.displayTimer) clearTimeout(this.displayTimer);

  if (state === 'connected') {
    this.displayedState.set('connected'); // recovery is shown immediately, no debounce
    needed
    return;
  }
  this.displayTimer = setTimeout(() => {
    if (this.rawState() !== 'connected') this.displayedState.set(state);
  }, 2000);
}
}

```

Additional points worth raising: cap `attempt` so `2 ** attempt` doesn't overflow before hitting `maxDelayMs` (the `Math.min` above already guards this). Add a visibility-change listener so backgrounded tabs don't keep retrying — reconnect immediately (attempt reset) on `document.visibilitychange` → visible, since mobile OSes often kill sockets when a tab is backgrounded. Consider a heartbeat/ping-pong on top of `onclose`, because some proxies keep a TCP connection "open" without actually delivering data — without an app-level heartbeat timeout, `onclose` may never fire for a half-dead connection.

Interviewer intent: Tests whether the candidate treats reconnection as a systems problem (backoff, jitter, thundering herd, UI debouncing) rather than a naive retry loop, and understands the difference between raw connection state and user-facing state.

Scenario 5: Merging live WebSocket updates into an already-paginated, cached REST list

Situation: A support-ticket inbox loads tickets via paginated REST (`GET /tickets?page=2`) and caches pages in a signal-based store. A WebSocket pushes `ticket-updated` and `ticket-created` events. New tickets need to appear at the top of page 1 in real time; updates to a ticket already visible on the currently viewed page need to patch in place; updates to a ticket that exists on a page the user hasn't loaded should not corrupt pagination.

Question: How do you architect the merge between the paginated cache and the live stream without breaking page boundaries or causing duplicate/missing rows?

Answer: Keep two structurally different stores and combine them at read time rather than mutating page arrays directly: a normalized entity map (`Map<id, Ticket>`, the "truth" for any ticket regardless of pagination) and a set of ordered ID lists per page (`Map<pageNumber, string[]>`, purely structural — which IDs belong on which page). Live updates only ever touch the entity map; pagination arrays are only touched by explicit REST fetches or a deliberate "new items available" banner action, never silently by the socket. This separation means an update to a ticket's status never has to know or care which page it's currently displayed on — every page simply looks up its IDs in the shared entity map.

```
import { Injectable, signal, computed } from '@angular/core';

interface Ticket { id: string; subject: string; status: string; updatedAt: number; }

@Injectable({ providedIn: 'root' })
export class TicketStore {
  private entities = signal<Map<string, Ticket>>(new Map());
  private pageIds = signal<Map<number, string[]>>(new Map());
  private newTicketBuffer = signal<Ticket[]>([]); // held back, not spliced into page 1 live

  getPage(page: number) {
    return computed(() => {
      const ids = this.pageIds().get(page) ?? [];
      const map = this.entities();
      return ids.map(id => map.get(id)).filter((t): t is Ticket => !!t);
    });
  }

  readonly newTicketsAvailable = computed(() => this.newTicketBuffer().length);

  onRestPageLoaded(page: number, tickets: Ticket[]): void {
    this.entities.update(map => {
      const next = new Map(map);
      for (const t of tickets) next.set(t.id, t);
      return next;
    });
    this.pageIds.update(map => new Map(map).set(page, tickets.map(t => t.id)));
  }

  // In-place patch for an existing entity – safe regardless of which page(s) reference it.
  onTicketUpdated(ticket: Ticket): void {
    this.entities.update(map => {
      const existing = map.get(ticket.id);
      if (!existing || ticket.updatedAt <= existing.updatedAt) return map; // ignore
      stale/unknown
      const next = new Map(map);
      next.set(ticket.id, ticket);
      return next;
    });
  }
}
```

```
// New tickets are queued for the user to pull in explicitly – never silently reshuffle
page 1.
onTicketCreated(ticket: Ticket): void {
  this.newTicketBuffer.update(list => [ticket, ...list]);
}

applyNewTickets(): void {
  const incoming = this.newTicketBuffer();
  this.entities.update(map => {
    const next = new Map(map);
    for (const t of incoming) next.set(t.id, t);
    return next;
  });
  this.pageIds.update(map => {
    const next = new Map(map);
    const page1 = next.get(1) ?? [];
    next.set(1, [...incoming.map(t => t.id), ...page1]);
    return next;
  });
  this.newTicketBuffer.set([]);
}
}
```

The critical design decision to call out in an interview: **structural changes to a paginated list (insert/reorder) must be an explicit, user-triggered action** ("3 new tickets — click to load"), while **in-place field updates to entities already known to the client can apply silently** because they don't shift anyone's scroll position or invalidate offsets. Conflating the two — letting a socket event silently splice a page array — is what causes duplicate rows, off-by-one pagination, and "the list jumped while I was reading it" complaints.

Interviewer intent: Tests whether the candidate can reason about the difference between normalized entity state and structural/positional state, and whether they know real-time inserts into a paginated view need explicit user consent, not silent injection.

Scenario 6: Collaborative document editing needs live cursors and presence without editing conflicts

Situation: You're building a Google-Docs-style collaborative text editor. Multiple users see each other's cursor positions and selections in real time, plus a small avatar stack of "who's currently viewing." The naive first attempt broadcasts the full document content on every keystroke, which is both bandwidth-heavy and produces edit conflicts when two users type at the same position simultaneously.

Question: Describe an architecture for presence (cursors, avatars) and content sync that avoids both bandwidth blow-up and lost edits, and how you'd structure this on the Angular side with signals.

Answer: Separate the two concerns entirely — they have different consistency requirements and update frequencies:

- **Presence (cursors, selections, "who's online")** is ephemeral, last-write-wins, and doesn't need conflict resolution — losing an intermediate cursor position is invisible to users. Broadcast small, throttled deltas (`{userId, position}`), not the whole roster.
- **Content edits** must never silently overwrite each other. The production-grade answer is CRDTs (e.g.,

Yjs) or Operational Transform, where each keystroke is sent as a small op (`insert(pos, char)` / `delete(pos, len)`), not a full-content snapshot — this makes concurrent edits at different positions merge cleanly and edits at the same position resolve deterministically instead of last-write-wins clobbering.

```
import { Injectable, signal, computed } from '@angular/core';

interface Presence { userId: string; name: string; color: string; cursorPos: number;
lastSeen: number; }
interface EditOp { userId: string; type: 'insert' | 'delete'; pos: number; text?: string;
len?: number; opId: string; }

@Injectable({ providedIn: 'root' })
export class CollabDocStore {
  private presenceMap = signal<Map<string, Presence>>(new Map());
  readonly activeUsers = computed(() => {
    const cutoff = Date.now() - 10_000; // treat silent-for-10s as "gone", not literally
    disconnected
    return [...this.presenceMap().values()].filter(p => p.lastSeen > cutoff);
  });

  private cursorSendTimer: ReturnType<typeof setTimeout> | null = null;
  private lastSentPos: number | null = null;

  // Throttle outbound cursor broadcasts – don't send on every pixel/keystroke movement.
  onLocalCursorMove(pos: number): void {
    if (this.cursorSendTimer) return; // already scheduled, coalesce
    this.cursorSendTimer = setTimeout(() => {
      this.cursorSendTimer = null;
      if (pos !== this.lastSentPos) {
        this.lastSentPos = pos;
        this.socket.send({ type: 'presence', cursorPos: pos });
      }
    }, 80); // ~12 updates/sec ceiling – fluid but not wasteful
  }

  onRemotePresence(p: Presence): void {
    this.presenceMap.update(map => new Map(map).set(p.userId, { ...p, lastSeen: Date.now()
  }));
  }

  // Content ops go through a CRDT/OT engine (e.g. Yjs) – the Angular layer only renders its
  output.
  // The store never applies raw remote text ops itself; it hands them to the CRDT doc,
  // which resolves ordering/conflicts, and re-derives the editor's displayed text from it.
  applyRemoteOp(op: EditOp, crdtDoc: { applyOp(op: EditOp): string }): void {
    const mergedText = crdtDoc.applyOp(op); // conflict resolution lives in the CRDT, not
    here
    this.docText.set(mergedText);
  }

  private docText = signal('');
```

```
readonly text = computed(() => this.docText());
}
```

Points to emphasize in the answer: (1) never diff-and-send full document content per keystroke — send ops. (2) Presence uses a `lastSeen` heartbeat + client-side staleness cutoff rather than relying solely on socket-disconnect events, because a user can go idle without disconnecting (switch tabs) and you still want their cursor to fade after inactivity. (3) Rolling your own OT is a well-known trap (correct OT transform functions are notoriously hard to get right for all op-pair combinations) — in a real interview, saying "I'd use Yjs/Automerger rather than hand-roll conflict resolution" is the senior answer. (4) Presence throttling (80ms) and content-op sending have very different urgency — cursors can lag slightly, character insertions generally should not be throttled at all (or only coalesced at the keystroke level, since users expect their own typing to feel instant locally and sync in the background).

Interviewer intent: Tests whether the candidate knows presence and content-sync are fundamentally different problems requiring different consistency models, and whether they reach for CRDT/OT rather than naively broadcasting full state on every change.

Scenario 7: A firehose of analytics events is overwhelming a live activity feed

Situation: An admin "live activity" panel shows events like "User X clicked button Y" streamed from a high-traffic production site — sometimes 500+ events/second during peak load. Rendering each as a new DOM row instantly crashes the tab.

Question: How do you rate-limit the UI so it stays useful (shows recent activity, feels "live") without trying to render every single event?

Answer: The fix is to decouple ingestion rate from render rate using a fixed-size ring buffer plus a sampling/aggregation window, rather than trying to render 1:1. Two complementary techniques: (1) cap the buffer so memory is bounded regardless of firehose volume, (2) aggregate bursts into summary rows ("47 clicks on Checkout in the last second") instead of 47 individual rows when the rate exceeds a threshold.

```
import { Injectable, signal } from '@angular/core';

interface ActivityEvent { type: string; label: string; ts: number; }
interface FeedRow { label: string; count: number; lastTs: number; }

@Injectable({ providedIn: 'root' })
export class ActivityFeedStore {
  private readonly maxRows = 100;
  private readonly flushIntervalMs = 500;
  private buffer: ActivityEvent[] = [];
  readonly rows = signal<FeedRow[]>([]);

  constructor() {
    setInterval(() => this.flush(), this.flushIntervalMs);
  }

  // called synchronously per incoming event – O(1), no signal write, no render triggered here.
}
```

```

onEvent(evt: ActivityEvent): void {
  this.buffer.push(evt);
  if (this.buffer.length > 5000) this.buffer.shift(); // hard safety cap on raw buffer
}

private flush(): void {
  if (this.buffer.length === 0) return;
  const batch = this.buffer;
  this.buffer = [];

  // Aggregate same-label bursts within this window into one row with a count.
  const grouped = new Map<string, FeedRow>();
  for (const evt of batch) {
    const existing = grouped.get(evt.label);
    if (existing) {
      existing.count++;
      existing.lastTs = evt.ts;
    } else {
      grouped.set(evt.label, { label: evt.label, count: 1, lastTs: evt.ts });
    }
  }

  this.rows.update(current => {
    const merged = [...grouped.values(), ...current];
    return merged.slice(0, this.maxRows); // bounded row count regardless of feed volume
  });
}

```

Design tradeoffs to discuss: a 500ms flush window means individual events are never shown one-by-one during a burst — the feed becomes "47 clicks in the last 0.5s" rather than 47 rows, which is the correct tradeoff for a human-readable activity panel (nobody can read 500 rows/sec anyway). If per-event detail truly matters (e.g., a debugging trace viewer), the better answer is virtual scrolling (`@angular/cdk/scrolling` or a signal-driven windowed list) over a capped in-memory ring buffer, rendering only the ~30 rows currently in the viewport, combined with the same buffer cap so memory doesn't grow unbounded. The `maxRows` cap on the `displayed` array (not just the raw buffer) is what actually protects the DOM — capping only the ingestion buffer but letting `rows` grow unbounded would still eventually degrade scroll performance.

Interviewer intent: Tests whether the candidate distinguishes "render every event" from "make the feed feel live," and whether they think about bounding both the buffer and the rendered list, not just one.

Scenario 8: A real-time notification badge needs to stay accurate across tabs and reconnects

Situation: A notification bell icon shows an unread count. Users report the count sometimes shows a stale number after reconnecting from a dropped socket, sometimes double-counts when the same tab is open twice, and sometimes doesn't decrement when the user reads a notification in another browser tab.

Question: Design the notification badge architecture end to end — count source of truth, cross-tab sync, and reconnect resync.

Answer: The badge count must never be purely additive client-side state (`count++` on each incoming push) because that drifts under duplicate deliveries, missed deliveries during a drop, and multi-tab scenarios. The count should always be derived from a set of notification IDs and their read/unread flags — a `computed()` over normalized entity state — with the server treated as the ultimate source of truth on every reconnect, and cross-tab sync handled via `BroadcastChannel` (or a shared `localStorage` event) rather than each tab keeping independent state.

```
import { Injectable, signal, computed, inject } from '@angular/core';

interface Notification { id: string; read: boolean; createdAt: number; }

@Injectable({ providedIn: 'root' })
export class NotificationStore {
  private notifications = signal<Map<string, Notification>>(new Map());
  readonly unreadCount = computed(() =>
    [...this.notifications().values()].filter(n => !n.read).length
  );

  private channel = new BroadcastChannel('notifications-sync');

  constructor() {
    // Cross-tab: any tab's read-action or new-notification is mirrored to sibling tabs
    // instantly.
    this.channel.onmessage = (e: MessageEvent) => {
      const { type, payload } = e.data;
      if (type === 'notification-read') this.markReadLocal(payload.id, /*broadcast*/ false);
      if (type === 'notification-added') this.addLocal(payload, /*broadcast*/ false);
    };
  }

  // Called once on socket connect/reconnect – reconciles against the server, doesn't trust
  // local state.
  async resyncFromServer(): Promise<void> {
    const serverList: Notification[] = await this.api.fetchAllNotifications();
    this.notifications.set(new Map(serverList.map(n => [n.id, n])));
  }

  onPush(n: Notification): void {
    this.addLocal(n, true);
  }

  private addLocal(n: Notification, broadcast: boolean): void {
    this.notifications.update(map => {
      if (map.has(n.id)) return map; // dedupe: same push delivered twice (e.g. tab + socket
      // retry)
      return new Map(map).set(n.id, n);
    });
    if (broadcast) this.channel.postMessage({ type: 'notification-added', payload: n });
  }

  markRead(id: string): void {
    this.markReadLocal(id, true);
  }
}
```

```

    this.api.markReadOnServer(id); // fire-and-forget-ish; server is still consulted on next
    resync
  }

  private markReadLocal(id: string, broadcast: boolean): void {
    this.notifications.update(map => {
      const n = map.get(id);
      if (!n || n.read) return map;
      const next = new Map(map);
      next.set(id, { ...n, read: true });
      return next;
    });
    if (broadcast) this.channel.postMessage({ type: 'notification-read', payload: { id } });
  }

  private api = inject(NotificationApi);
}

// Placeholder type for illustration
declare class NotificationApi {
  fetchAllNotifications(): Promise<Notification[]>;
  markReadOnServer(id: string): Promise<void>;
}

```

Key architectural decisions to call out: (1) the badge count is *always* `computed()` from the notification set, never an independently incremented counter — this makes it self-correcting the moment the underlying set is accurate. (2) On every reconnect, do a full resync from the server rather than assuming the socket delivered everything that happened while disconnected — sockets dropping means missed pushes are a certainty, not an edge case. (3) `BroadcastChannel` keeps same-origin tabs consistent without a server round-trip, which fixes both the double-count-across-two-tabs bug (each tab adds once, dedupes by ID) and the cross-tab "mark as read" bug. (4) Deduping by notification ID handles at-least-once delivery semantics that most real-time transports actually provide (despite looking like exactly-once in the happy path).

Interviewer intent: Tests whether the candidate treats a "simple counter" UI as something requiring the same rigor as any other distributed-state problem — dedup, resync-on-reconnect, and cross-tab consistency — rather than a naive incrementing integer.

Scenario 9: Effect() is firing infinitely when wiring a signal to a WebSocket send

Situation: A developer writes `effect(() => this.socket.send(JSON.stringify(this.filters())))` to push filter changes to the server in real time for a live-filtered dataset. In production this causes a runaway loop of socket messages, and occasionally an `NG0600` write-after-read error in the console.

Question: What's actually going wrong here, and how should signal-to-socket synchronization be structured with `effect()`?

Answer: Two separate bugs are likely combined. First, if the server's acknowledgment or echoed state is fed back into the same `filters` signal (common when the socket handler does `this.filters.set(serverEcho)`), the effect refires because `filters()` changed, sends again, gets echoed again — an infinite ping-pong. Second, `NG0600` specifically means a signal was written during the same synchronous change-detection pass that's still computing reads (e.g., writing to a signal that's also read by the *same* effect, or by a template currently rendering) — `effect()` bodies should generally treat signal writes as a side effect performed carefully, not casually mix reads-that-drive-the-effect with writes-to-the-same-signal.

The correct structure: keep "local filter state the user controls" and "last-acknowledged-by-server state" as two distinct signals, and only ever send when the *local* one changes, never write the local one from the server echo.

```
import { Injectable, signal, effect, EnvironmentInjector, runInInjectionContext } from
  '@angular/core';

interface Filters { status: string; assignee: string | null; }

@Injectable({ providedIn: 'root' })
export class LiveFiltersSync {
  readonly filters = signal<Filters>({ status: 'open', assignee: null });
  private lastSentJson: string | null = null;

  constructor(private injector: EnvironmentInjector) {
    runInInjectionContext(this.injector, () => {
      effect(() => {
        const json = JSON.stringify(this.filters());
        // Guard: only send when the value actually differs from what we last transmitted -
        // prevents re-sending (and any echo-triggered loop) on unrelated re-evaluations.
        if (json !== this.lastSentJson) {
          this.lastSentJson = json;
          this.socket.send(json);
        }
      });
    });
  }

  // Server confirmations update a SEPARATE signal - never feed the ack back into `filters`.
  private confirmedByServer = signal<Filters | null>(null);
  onServerAck(filters: Filters): void {
    this.confirmedByServer.set(filters);
  }

  private socket = { send: (_: string) => {} }; // illustrative
}
```

Additional reasoning: `effect()` is meant for synchronizing signals with the *outside world* (DOM, storage, sockets) — it is explicitly not meant to be a general reactive pipeline where effects write to signals that other effects (or the same effect) read, because that reintroduces the exact glitches/cascades Angular's signal model is designed to avoid. The `lastSentJson` guard is a manual "distinctUntilChanged" — in an RxJS-flavored answer you'd mention `toObservable(this.filters).pipe(distinctUntilChanged(), debounceTime(150))` as an alternative that also naturally adds debounce for rapid filter changes (e.g., a user toggling three

checkboxes quickly should ideally send one socket message, not three) — worth mentioning `debounceTime` as the production-grade addition here.

Interviewer intent: Tests understanding of `effect()`'s intended use (syncing signals to the outside world) versus misuse (feedback loops via echoed server state), and familiarity with `NG0600` as a real diagnostic signal, not just trivia.

Scenario 10: Multiple browser tabs open the same live dashboard and each opens its own WebSocket

Situation: A user regularly has 4-5 tabs of the same real-time dashboard open. Each tab independently opens a WebSocket connection to the same backend, multiplying server-side connection count and causing rate-limit warnings from the backend team.

Question: How would you architect a single shared real-time connection across multiple tabs of the same app, and what fallback do you need if that sharing mechanism isn't available?

Answer: Use a `SharedWorker` to own the single WebSocket connection, with each tab connecting to the `sharedworker` (which the browser automatically deduplicates to one instance per origin) instead of opening its own socket. Tabs communicate with the worker via `MessagePort`; the worker fans out incoming messages to every connected tab and forwards outbound sends from any tab to the single upstream socket. Because `sharedworker` isn't universally available in every context (some corporate/embedded browser environments, and it doesn't exist in Web Workers-in-workers chains), a graceful fallback is an "elected leader tab" pattern using the Web Locks API or a `BroadcastChannel` + heartbeat election, where one tab holds the real socket and relays via `BroadcastChannel` to its siblings.

```
// realtime-shared-worker.ts – runs inside the SharedWorker
const ports: MessagePort[] = [];
let socket: WebSocket | null = null;

function ensureSocket(url: string) {
  if (socket && socket.readyState <= WebSocket.OPEN) return;
  socket = new WebSocket(url);
  socket.onmessage = (evt) => {
    for (const port of ports) port.postMessage({ type: 'data', payload: evt.data });
  };
}

// @ts-ignore – SharedWorkerGlobalScope
onconnect = (event: MessageEvent) => {
  const port = event.ports[0];
  ports.push(port);
  port.onmessage = (e) => {
    if (e.data.type === 'init') ensureSocket(e.data.url);
    if (e.data.type === 'send' && socket) socket.send(e.data.payload);
  };
  port.start();
};
```

```
// realtime-connection.service.ts – used by each tab
```

```
import { Injectable, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class SharedRealtimeConnection {
  readonly lastMessage = signal<unknown>(null);
  private port?: MessagePort;

  connect(url: string): void {
    const worker = new SharedWorker(new URL('./realtime-shared-worker.ts',
import.meta.url));
    this.port = worker.port;
    this.port.onmessage = (e) => {
      if (e.data.type === 'data') this.lastMessage.set(JSON.parse(e.data.payload));
    };
    this.port.start();
    this.port.postMessage({ type: 'init', url }); // idempotent: worker only opens socket if
needed
  }

  send(payload: unknown): void {
    this.port?.postMessage({ type: 'send', payload: JSON.stringify(payload) });
  }
}
```

Tradeoffs worth surfacing: `SharedWorker` support is solid in Chromium/Firefox but historically inconsistent in some WebKit/Safari builds and unavailable in some mobile browsers — a production system needs the leader-election `BroadcastChannel` fallback, or should feature-detect `typeof SharedWorker !== 'undefined'` and degrade to "every tab has its own socket" as the last-resort fallback rather than crashing. Also worth mentioning: closing tabs must be handled — the worker should track live ports and close the upstream socket only when the last port disconnects (`port.onmessage` won't fire a "tab closed" event directly; a periodic heartbeat ping/pong between tab and worker, or listening for `unload / pagehide` to send an explicit "goodbye" message, is needed to detect tab closure reliably).

Interviewer intent: Tests awareness of `SharedWorker` as a real solution to multi-tab connection duplication, and whether the candidate proactively surfaces browser-support caveats and a fallback rather than presenting one API as a silver bullet.

Scenario 11: A live sports-score widget needs smooth number transitions without layout thrash

Situation: A live score ticker updates a large numeric score display multiple times per minute. The instant DOM text swap looks jarring, but a naive CSS transition on every possible child causes visible layout recalculation stutter on lower-end devices during bursts of updates.

Question: How do you animate frequent real-time value changes smoothly and cheaply?

Answer: The key perf principle is to animate only compositor-friendly properties (`transform`, `opacity`) and never trigger layout/reflow-inducing properties (width, top/left, font-size) on a value that changes many times per minute. For a "count up/down" visual effect, animate via `requestAnimationFrame` interpolation into a signal that drives only text content (no layout impact from text changes if the container has a fixed width), and

use CSS `transform: scale()` + `opacity` for a "pulse" flash on update rather than restyling.

```
import { Component, input, effect, signal, ChangeDetectionStrategy } from '@angular/core';

@Component({
  selector: 'app-live-score',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<span class="score" [class.pulse]="pulsing()">{{ displayValue() }}</span>`,
  styles: [
    `.score { display: inline-block; min-width: 3ch; text-align: right; } /* fixed width avoids reflow */`
    .pulse { animation: flash 300ms ease-out; }
    @keyframes flash {
      0% { transform: scale(1.15); opacity: 0.6; }
      100% { transform: scale(1); opacity: 1; }
    }
  ],
})
export class LiveScoreComponent {
  value = input.required<number>();
  displayValue = signal(0);
  pulsing = signal(false);

  private raf?: number;

  constructor() {
    effect(() => {
      const target = this.value();
      this.animateTo(target);
    });
  }

  private animateTo(target: number): void {
    const start = this.displayValue();
    if (start === target) return;
    const startTime = performance.now();
    const duration = 250;

    this.pulsing.set(true);
    if (this.raf) cancelAnimationFrame(this.raf);

    const step = (now: number) => {
      const t = Math.min(1, (now - startTime) / duration);
      this.displayValue.set(Math.round(start + (target - start) * t));
      if (t < 1) {
        this.raf = requestAnimationFrame(step);
      } else {
        this.pulsing.set(false);
      }
    };
    this.raf = requestAnimationFrame(step);
  }
}
```

```
}
```

Reasoning to include: (1) `transform/opacity` animations run on the compositor thread and don't trigger layout or paint of surrounding elements, which is why they stay smooth even under CPU pressure from the message-processing side of the app. (2) Fixing `min-width` on the score span prevents the text-width change (e.g., "9" → "10") from reflowing sibling elements. (3) Cancelling the previous `requestAnimationFrame` before starting a new one prevents two overlapping animations from fighting over `displayValue` if a new score arrives mid-animation — a very real scenario in a fast-moving game. (4) `OnPush` plus a signal-driven template means Angular's change detection is only invoked when `displayValue/pulsing` actually change, and the RAF loop itself doesn't depend on `zone.js` at all — it works identically in a zoneless app.

Interviewer intent: Tests whether the candidate connects real-time UI update frequency to browser rendering-pipeline costs (layout vs. compositor-only properties), not just Angular-level optimization.

Scenario 12: Deciding between WebSocket, SSE, and polling for a real-time feature

Situation: The team is starting a new "live order status" feature (server → client only, no client → server real-time messages needed) and a separate "multiplayer cursor" feature (bidirectional, low latency required). A junior developer proposes using WebSockets for both because "it's the most real-time option."

Question: Push back or agree — how do you choose the transport for each feature, and what does that choice mean for the Angular service layer design?

Answer: WebSockets are the right (arguably only sensible) choice for the bidirectional low-latency cursor feature, but Server-Sent Events (SSE) is usually the better fit for the order-status feature: it's unidirectional (matches the actual requirement exactly), rides plain HTTP (works through more corporate proxies/load balancers without special config, gets automatic reconnection built into the browser's `EventSource` API, and is simpler to scale/debug since it's just a long-lived HTTP response). Reaching for WebSockets everywhere adds needless complexity (custom reconnect logic, custom message framing, no built-in `Last-Event-ID` resume support) for features that don't need bidirectionality.

```
import { Injectable, signal, NgZone, inject } from '@angular/core';

interface OrderStatus { orderId: string; status: string; }

@Injectable({ providedIn: 'root' })
export class OrderStatusFeed {
  readonly status = signal<OrderStatus | null>(null);
  private es?: EventSource;

  connect(orderId: string): void {
    // EventSource gives free auto-reconnect + Last-Event-ID resume – no custom backoff code
    // needed here.
    this.es = new EventSource(`/api/orders/${orderId}/status-stream`);
    this.es.onmessage = (evt) => {
      this.status.set(JSON.parse(evt.data));
    };
    this.es.onerror = () => {
```

```
// Browser will auto-retry SSE by default; only act if we want custom UX (e.g. a
banner).
};
}

disconnect(): void {
  this.es?.close();
}
}
```

```
// Cursor feature: genuinely bidirectional and latency-sensitive => WebSocket is correct
here.
@Injectables({ providedIn: 'root' })
export class CursorSocketFeed {
  private ws?: WebSocket;
  readonly remoteCursor = signal<{ x: number; y: number } | null>(null);

  connect(url: string): void {
    this.ws = new WebSocket(url);
    this.ws.onmessage = (evt) => this.remoteCursor.set(JSON.parse(evt.data));
  }

  sendCursor(x: number, y: number): void {
    this.ws?.send(JSON.stringify({ x, y }));
  }
}
```

Decision framework to lay out verbally: choose SSE when the data flow is strictly server→client, HTTP/2 multiplexing and proxy compatibility matter, and you want free reconnection/resume semantics. Choose WebSocket when you need true bidirectional low-latency messaging (typing indicators, cursors, game state) or non-text/binary framing. Choose plain polling (possibly long-polling) only when infrastructure genuinely can't support persistent connections (some enterprise proxies) or update frequency is low enough (every 30-60s) that the operational simplicity of "just an HTTP GET on an interval" outweighs any real-time benefit. On the Angular side, the key architectural point is that both transports should be hidden behind a signal-exposing service interface (`readonly status = signal(...)`) so components never know or care which transport is underneath — that boundary is what lets you swap SSE for WebSocket later without touching consuming components.

Interviewer intent: Tests whether the candidate can justify transport choice by actual requirements (directionality, reconnection needs, infra constraints) instead of defaulting to "WebSocket is most real-time so use it everywhere," and whether they abstract transport behind a stable service API.

Scenario 13: A live leaderboard re-sorts and causes jarring row jumps on every update

Situation: A real-time leaderboard re-sorts its list every time any player's score changes (which happens dozens of times per minute). Rows visibly teleport up and down the list every update, making it hard to track any single player, and `*ngFor / @for` without `track` was causing full DOM node recreation on every re-sort.

Question: How do you make frequent re-sorting both computationally efficient and visually comprehensible to the user?

Answer: Two separate problems: correctness/efficiency of the re-render (fixed by proper `track` usage so Angular reuses DOM nodes rather than destroying/recreating them), and UX comprehensibility of frequent reordering (fixed by animating position changes with FLIP-style transitions, and optionally debouncing re-sort frequency so a player who briefly overtakes another and immediately loses the lead again doesn't visibly flicker).

```
import { Component, signal, computed, ChangeDetectionStrategy } from '@angular/core';

interface Player { id: string; name: string; score: number; }

@Component({
  selector: 'app-leaderboard',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @for (player of sortedPlayers(); track player.id) {
      <div class="row" [style.order]="$index">
        {{ player.name }} - {{ player.score }}
      </div>
    }
  `,
  styles: [
    .row { transition: transform 300ms ease; } /* combined with track, enables smooth reflow perception */
  ],
})
export class LeaderboardComponent {
  private rawPlayers = signal<Player[]>([]);
  private lastSortAt = 0;
  private readonly resortMinIntervalMs = 1000; // cap re-sort frequency, not per-score-update frequency

  // Scores update immediately (values are always fresh); ORDER only recomputes at most once/sec.
  private frozenOrder = signal<string[]>([]);

  sortedPlayers = computed(() => {
    const byId = new Map<string, Player>(this.rawPlayers().map(p => [p.id, p]));
    return this.frozenOrder()
      .map(id => byId.get(id))
      .filter((p): p is Player => !!p);
  });

  onScoreUpdate(update: Player): void {
    this.rawPlayers.update(list => {
      const next = list.map(p => (p.id === update.id ? update : p));
      return next.some(p => p.id === update.id) ? next : [...next, update];
    });
    this.maybeResort();
  }
}
```

```

    }

    private maybeResort(): void {
      const now = Date.now();
      if (now - this.lastSortAt < this.resortMinIntervalMs) return;
      this.lastSortAt = now;
      const sorted = [...this.rawPlayers()].sort((a, b) => b.score - a.score);
      this.frozenOrder.set(sorted.map(p => p.id));
    }
  }
}

```

Key ideas: `track player.id` (using the built-in `@for` tracking, the modern replacement for `trackBy`) ensures Angular reuses the same DOM element for a given player across re-sorts instead of destroying and recreating it — this alone is what makes a CSS `transition: transform` on reordering actually visible/smooth, since a destroyed-and-recreated element can't transition from its old position. Separating "score value" (always live) from "row order" (re-derived at most once per second) directly addresses the "jarring jumps" complaint — the numbers update in real time (feels live) while the visual position only reshuffles on a controlled cadence (feels stable and readable). For true FLIP (First-Last-Invert-Play) animated reordering you'd additionally capture each row's bounding rect before and after the reorder and animate the transform delta — a well-known technique often implemented via the Angular CDK's drag-drop utilities or a small custom directive, worth naming even if not coding it out in full.

Interviewer intent: Tests whether the candidate connects `track` / `trackBy` correctness to animation feasibility, and whether they think to decouple value freshness from positional stability for UX reasons, not just performance reasons.

Scenario 14: Determining whether a signal write happened locally or from a real-time echo, to avoid animation on your own edits

Situation: In a live-updating price list, every price change triggers a "flash" highlight animation. Users complain that when *they themselves* change a filter that causes their own optimistic local update, it flashes as if it were a remote update — which feels noisy and wrong; only genuinely remote changes should flash.

Question: How do you distinguish "this signal changed because of my own action" from "this signal changed because of an incoming real-time event," when both ultimately call the same `.set()`?

Answer: A signal's value has no memory of *why* it changed — so the "why" must be carried explicitly as metadata alongside the value, not inferred after the fact. The cleanest approach is to store an object `{ value, source }` rather than a bare value, or maintain a parallel "last change source" signal that the flash-animation logic reads.

```

import { Injectable, signal, computed, effect } from '@angular/core';

type ChangeSource = 'local' | 'remote';
interface PricedItem { id: string; price: number; }
interface Tagged<T> { value: T; source: ChangeSource; changedAt: number; }

@Injectable({ providedIn: 'root' })

```

```

export class PriceListStore {
  private items = signal<Map<string, Tagged<PricedItem>>>(new Map());

  readonly displayItems = computed(() =>
    [...this.items().values()].map(t => t.value)
  );

  // Component-level flash logic reads THIS, not just the price value.
  shouldFlash(id: string, withinMs = 400): boolean {
    const entry = this.items().get(id);
    if (!entry || entry.source !== 'remote') return false;
    return Date.now() - entry.changedAt < withinMs;
  }

  applyLocalChange(item: PricedItem): void {
    this.items.update(map => {
      const next = new Map(map);
      next.set(item.id, { value: item, source: 'local', changedAt: Date.now() });
      return next;
    });
  }

  applyRemoteChange(item: PricedItem): void {
    this.items.update(map => {
      const next = new Map(map);
      next.set(item.id, { value: item, source: 'remote', changedAt: Date.now() });
      return next;
    });
  }
}

```

The row component checks `store.shouldFlash(item.id)` (perhaps via a small `computed()` per row, or a signal-based interval that re-evaluates the time window) and applies the `.pulse` class only for remote-sourced, recent changes. Alternative approaches worth mentioning: (a) a short-lived "recently touched by me" `Set<string>` of IDs that the local-change path adds to and automatically evicts after a timeout, checked with negation (`!recentlyLocal.has(id) ⇒ eligible to flash`) — simpler when you only need a boolean, not full provenance; (b) tagging outbound messages with a client-generated `originId` and having the server echo it back, so the *client* can recognize "this remote event is actually my own optimistic update being confirmed" and suppress the flash for exactly that round-trip — necessary in architectures where all writes (even your own) come back through the same real-time channel rather than being applied optimistically client-side first.

Interviewer intent: Tests whether the candidate recognizes that "why did this value change" is not recoverable from the value alone and must be modeled explicitly as data, a subtle but common real-time UI requirement (avoiding self-triggered animations/notifications).

Scenario 15: A signal-based store needs to unsubscribe from a WebSocket exactly when the last consuming component is destroyed

Situation: A `LiveMetricsService` opens a WebSocket connection when first used and should close it when no component is displaying real-time metrics anymore (to save server resources), but should NOT close it if one dashboard component is destroyed while a different one (say, a modal also showing the same metric) is still alive.

Question: Design reference-counted connection lifecycle management so the socket opens/closes correctly regardless of how many components share it and in what order they mount/unmount.

Answer: This is a reference-counting problem, not a "component destroy = close socket" problem. Each consumer registers interest and gets back an unsubscribe/release function (mirroring `DestroyRef`'s `onDestroy` callback pattern); the service only tears down the connection when the count returns to zero.

```
import { Injectable, signal, DestroyRef, inject } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class LiveMetricsService {
  readonly metrics = signal<Record<string, number> | null>(null);
  private refCount = 0;
  private ws?: WebSocket;

  // Call from a component's constructor; automatically releases on that component's
  // destroy.
  acquire(): void {
    const destroyRef = inject(DestroyRef); // must be called in an injection context by the
    caller
    this.refCount++;
    if (this.refCount === 1) this.openConnection();
    destroyRef.onDestroy(() => this.release());
  }

  private release(): void {
    this.refCount = Math.max(0, this.refCount - 1);
    if (this.refCount === 0) this.closeConnection();
  }

  private openConnection(): void {
    this.ws = new WebSocket('wss://api.example.com/metrics');
    this.ws.onmessage = (evt) => this.metrics.set(JSON.parse(evt.data));
  }

  private closeConnection(): void {
    this.ws?.close();
    this.ws = undefined;
    this.metrics.set(null);
  }
}
```

```
// Consuming component
import { Component, inject } from '@angular/core';

@Component({ selector: 'app-metrics-widget', standalone: true, template: `{{
service.metrics() | json }}` })
export class MetricsWidgetComponent {
  service = inject(LiveMetricsService);
  constructor() {
    this.service.acquire(); // registers interest + auto-releases on this component's
destroy
  }
}
```

Important nuances to raise: `DestroyRef` must be injected within the constructor/injection context of the *consuming* component (or passed in), not inside the service's own constructor, since the service is a singleton with root-level lifetime and its own `DestroyRef` would only fire on app shutdown, not per-component. A subtlety worth mentioning proactively: if a component mounts, unmounts, and a new one mounts again within the same tick (e.g., router navigating between two routes that both show the widget), naive teardown could open/close the socket rapidly; a small "grace period" (delay actual close by ~2s, cancel the pending close if `refCount` goes back above 0) avoids needless reconnect churn for that common transition pattern — the same debounce idea as Scenario 4's connection-state display, applied to teardown timing instead of UI display timing.

Interviewer intent: Tests whether the candidate reaches for reference counting (rather than binding resource lifetime to any single component's lifecycle) when a shared real-time resource has multiple simultaneous consumers, and correct usage of `DestroyRef` in a service context.

Scenario 16: Real-time updates need to respect user permissions that can change mid-session

Situation: In a multi-tenant real-time dashboard, a user's role/permissions can be revoked mid-session by an admin (e.g., downgraded from "manager" to "viewer"). Currently the WebSocket keeps pushing manager-only data (e.g., other employees' salary figures) to the client even after the permission change, because the socket subscription was established once at login and never re-evaluated.

Question: How do you ensure real-time data streams respect authorization changes that happen mid-session, not just at initial connection?

Answer: Authorization for a persistent connection can't be a one-time check at handshake — it has to be enforced continuously, ideally server-side (never trust the client to self-censor), with the client-side store treating permission signals as first-class reactive state that can prune/re-filter already-received real-time data. Two layers: (1) the server must re-validate permissions on every message it's about to push (or subscribe the client to permission-scoped channels/rooms it can revoke membership from), and (2) the client should also defensively react to a permission-downgrade event by immediately clearing any cached sensitive data client-side, both as defense-in-depth and to fix the UI within the same tick rather than waiting for the next unrelated re-render.

```
import { Injectable, signal, computed, effect } from '@angular/core';
```

```

interface Permissions { canViewSalaries: boolean; role: string; }
interface EmployeeMetric { employeeId: string; salary?: number; hoursLogged: number; }

@Injectables({ providedIn: 'root' })
export class TeamDashboardStore {
  readonly permissions = signal<Permissions>({ canViewSalaries: false, role: 'viewer' });
  private rawMetrics = signal<Map<string, EmployeeMetric>>(new Map());

  // Client-side redaction is a UX/defense-in-depth layer – the server must ALSO stop
  // sending
  // salary data once permission is revoked; this alone is not the security boundary.
  readonly visibleMetrics = computed(() => {
    const perms = this.permissions();
    return [...this.rawMetrics().values()].map(m =>
      perms.canViewSalaries ? m : { ...m, salary: undefined }
    );
  });

  constructor() {
    // React to permission downgrades in real time: purge sensitive cached data immediately.
    effect(() => {
      const perms = this.permissions();
      if (!perms.canViewSalaries) {
        this.rawMetrics.update(map => {
          const next = new Map(map);
          for (const [id, metric] of next) next.set(id, { ...metric, salary: undefined });
          return next;
        });
      }
    });
  }

  onPermissionsChanged(perms: Permissions): void {
    this.permissions.set(perms);
    // Also tell the server-side subscription to re-scope the channel/room for this
    // connection –
    // the socket subscription itself should be re-issued, not just filtered on the client.
    this.socket.send({ type: 'resubscribe', role: perms.role });
  }

  onMetricUpdate(metric: EmployeeMetric): void {
    this.rawMetrics.update(map => new Map(map).set(metric.employeeId, metric));
  }

  private socket = { send: (_: unknown) => {} };
}

```

Points to stress in the answer: client-side filtering (the `computed()` redaction) is a UX nicety, never the actual security boundary — if the raw salary value ever reached the browser's memory/network tab even briefly before being filtered in the view, that's a real data leak, so the *server* must stop including that field in pushed payloads the instant permission is revoked (ideally by moving the connection's subscription to a differently-scoped channel/room, not by relying on a client-honored flag). The client-side effect exists so the UI updates

instantly rather than waiting for the next unrelated event to trigger a re-render, and so any data that leaked through before revocation propagated is proactively scrubbed from the in-memory store.

Interviewer intent: Tests whether the candidate understands that authorization for persistent connections is continuous, not a one-time handshake check, and won't mistake client-side filtering for an actual security control.

Scenario 17: A live chart needs to show a smooth trend line from noisy per-second sensor data

Situation: An IoT dashboard streams raw sensor readings every ~200ms with significant jitter/noise. Plotting every raw point makes the line chart look like static, and re-rendering the whole chart component on every point causes visible CPU spikes on a page with 8 concurrent charts.

Question: How do you process a noisy high-frequency stream into a chart-friendly signal, and how do you keep 8 independent charts from competing for CPU on each update?

Answer: Apply a smoothing/downsampling transform (e.g., a simple moving average or exponential moving average) between the raw stream and the value the chart actually consumes, and batch chart redraws so 8 charts don't each trigger independent synchronous work — instead, all charts read from signals and get updated together on a single coalesced tick, similar to Scenario 1's per-frame flush but applied per-chart-series.

```
import { Injectable, signal, computed } from '@angular/core';

interface Reading { sensorId: string; value: number; ts: number; }

@Injectable({ providedIn: 'root' })
export class SensorChartStore {
  private readonly windowSize = 5; // simple moving average window
  private rawHistory = new Map<string, number[]>();
  private smoothed = new Map<string, ReturnType<typeof signal<number[]>>>();
  private pending = new Set<string>();
  private framesScheduled = false;

  getSeries(sensorId: string) {
    if (!this.smoothed.has(sensorId)) this.smoothed.set(sensorId, signal<number[]>([]));
    return this.smoothed.get(sensorId)!;
  }

  onReading(reading: Reading): void {
    const history = this.rawHistory.get(reading.sensorId) ?? [];
    history.push(reading.value);
    if (history.length > this.windowSize) history.shift();
    this.rawHistory.set(reading.sensorId, history);
    this.pending.add(reading.sensorId);

    if (!this.framesScheduled) {
      this.framesScheduled = true;
      requestAnimationFrame(() => this.flush());
    }
  }
}
```

```

}

// One coalesced pass updates ALL 8 charts' signals together, once per frame,
// instead of each raw message independently nudging its own chart.
private flush(): void {
  this.framesScheduled = false;
  for (const sensorId of this.pending) {
    const history = this.rawHistory.get(sensorId) ?? [];
    const avg = history.reduce((a, b) => a + b, 0) / (history.length || 1);
    const series = this.getSeries(sensorId);
    series.update(points => [...points.slice(-199), avg]); // cap chart history to 200
  }
  this.pending.clear();
}
}

```

Reasoning points: a moving average trades a small amount of latency (the displayed value lags the truly instantaneous raw reading by half the window) for a dramatically more readable trend line — worth explicitly naming this as a deliberate accuracy/readability tradeoff, and mentioning that an exponential moving average ($\text{ema} = \alpha * \text{new} + (1 - \alpha) * \text{ema}$) is often preferred over a simple windowed average because it doesn't require storing/shifting an array and reacts a bit faster to genuine trend changes while still damping noise. Coalescing all 8 sensors' updates into a single `requestAnimationFrame` callback (rather than 8 independent per-sensor RAF loops) means the browser does one layout/paint pass per frame for all charts combined, not up to 8 competing passes — this is the same principle as Scenario 1 (frame-based coalescing) but applied across multiple independent data series sharing the same rendering budget. Capping stored history per series (`slice(-199)`) keeps memory and per-frame array-copy cost bounded regardless of how long the dashboard has been open.

Interviewer intent: Tests whether the candidate thinks about both statistical smoothing (signal processing, not just Angular mechanics) and cross-component render coalescing when multiple independent real-time visualizations share a page.

Scenario 18: Testing a component that depends on a live WebSocket feed

Situation: A code reviewer flags that a component's unit tests spin up a real (mocked at the network layer) WebSocket connection, use `fakeAsync / tick()` to simulate message arrival, and are flaky — sometimes passing, sometimes timing out, depending on machine speed.

Question: What's the correct testing strategy for components/services built around real-time signal stores, and why does the current approach produce flakiness?

Answer: The flakiness comes from testing through the transport (WebSocket + its async event loop interactions) instead of testing the store's public API directly. The transport-facing code (open socket, parse frames, handle reconnect) should be a thin, separately-tested adapter; the store's business logic (merge, dedupe, ordering, optimistic patches) should be pure enough to unit-test synchronously by calling its public methods directly with plain objects — no fake timers, no real socket mock, no `tick()` needed at all for that layer.

```
// live-metrics.store.spec.ts – tests the STORE directly, no socket/transport involved.
import { TestBed } from '@angular/core/testing';
import { DriverLocationStore } from '../driver-location.store';

describe('DriverLocationStore', () => {
  let store: DriverLocationStore;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    store = TestBed.inject(DriverLocationStore);
  });

  it('ignores a stale update that arrives after a newer one (out-of-order transport)', () => {
    store.applyUpdate({ driverId: 'd1', lat: 1, lng: 1, serverTs: 200 });
    store.applyUpdate({ driverId: 'd1', lat: 0, lng: 0, serverTs: 100 }); // arrives late,
    // older ts

    expect(store.locations().get('d1')?.serverTs).toBe(200); // stale update correctly
    // rejected
  });

  it('accepts a genuinely newer update', () => {
    store.applyUpdate({ driverId: 'd1', lat: 1, lng: 1, serverTs: 200 });
    store.applyUpdate({ driverId: 'd1', lat: 2, lng: 2, serverTs: 300 });

    expect(store.locations().get('d1')?.serverTs).toBe(300);
  });
});
```

```
// realtime-connection.spec.ts – tests the TRANSPORT adapter in isolation, with an injected
// fake.
import { RealtimeConnection } from '../realtime-connection.service';

class FakeWebSocket {
  onopen?: () => void;
  onclose?: () => void;
  onmessage?: (e: MessageEvent) => void;
  close() { this.onclose?.(); }
  triggerOpen() { this.onopen?.(); }
}

it('transitions to connected only after the socket actually opens', () => {
  const fakeSocket = new FakeWebSocket();
  const connection = new RealtimeConnection(() => fakeSocket as unknown as WebSocket);

  connection.connect('wss://example.test');
  expect(connection.displayedState()).toBe('connecting');

  fakeSocket.triggerOpen();
  expect(connection.displayedState()).toBe('connected');
});
```

The core lesson: because the store exposes plain synchronous methods (`applyUpdate` , `onTicketUpdated` , etc.) operating on signals, tests can call them directly with hand-crafted payloads and assert on `signal()` reads immediately — no `fakeAsync` , no `tick()` , no timing dependency, hence zero flakiness. The transport adapter (whatever opens the real `WebSocket` / `EventSource`) should accept an injectable socket factory (as shown, a constructor parameter or an injected token) so tests can substitute a fully-controllable fake instead of a real or lightly-mocked browser API, and tests for it stay narrowly scoped to connection-state transitions rather than business logic. This separation (dumb transport adapter vs. pure/testable store logic) is generally the same architecture that makes the real-time system maintainable in production, not just testable — it's the same reasoning as Scenarios 2, 3, and 5 which all put transport-agnostic merge/ordering/patch logic in the store layer.

Interviewer intent: Tests whether the candidate can identify that flaky async tests are often a symptom of a leaky architecture (business logic entangled with transport timing) rather than reaching first for "add more `tick()` calls" or longer timeouts.

Scenario 19: A "typing indicator" feature needs to expire automatically without an explicit "stopped typing" event

Situation: A chat app shows "Alice is typing..." driven by a `typing-start` socket event. Occasionally the corresponding `typing-stop` event never arrives (dropped message, or the user's tab crashed mid-type), leaving a permanently stuck "is typing..." indicator until page reload.

Question: Design a typing-indicator mechanism that self-heals from a missing "stop" event without requiring the server to be perfectly reliable.

Answer: Never rely on a paired start/stop event model alone for anything ephemeral and time-bound — treat "is typing" as a TTL-based fact (true only until an expiry timestamp) that expires automatically client-side regardless of whether a "stop" event ever arrives, exactly the same principle as a lease/heartbeat in distributed systems. The server should periodically re-send `typing-start` while the user keeps typing (a heartbeat, not a one-shot event), and the client sets/refreshes a short expiry timer on each receipt.

```
import { Injectable, signal, computed } from '@angular/core';

interface TypingState { userId: string; expiresAt: number; }

@Injectable({ providedIn: 'root' })
export class TypingIndicatorStore {
  private typingUsers = signal<Map<string, TypingState>>>(new Map());
  private readonly ttlMs = 4000; // must be shorter than any plausible network partition
  tolerance need
  private tickHandle = signal(0); // forces periodic re-evaluation of expiry independent of
  new events

  constructor() {
    setInterval(() => this.tickHandle.update(n => n + 1), 1000); // sweep for expired
    entries every 1s
  }

  readonly activeTypers = computed(() => {
```

```

    this.tickHandle(); // dependency: re-run this computed on every sweep tick, not just on
new events
    const now = Date.now();
    return [...this.typingUsers().values()].filter(t => t.expiresAt > now).map(t =>
t.userId);
    });

    // Called on EVERY typing-start heartbeat from the server, not just the first one.
onTypingHeartbeat(userId: string): void {
    this.typingUsers.update(map =>
        new Map(map).set(userId, { userId, expiresAt: Date.now() + this.ttlMs })
    );
}

    // Still honor an explicit stop event if it DOES arrive – removes immediately rather than
waiting for TTL.
onTypingStop(userId: string): void {
    this.typingUsers.update(map => {
        const next = new Map(map);
        next.delete(userId);
        return next;
    });
}
}

```

Why this is robust: the indicator disappears within `ttlMs` of the last heartbeat no matter what happens to the connection, the sender's tab, or a dropped "stop" message — there is no failure mode (short of the sweep interval itself failing) that leaves it stuck forever. The explicit `onTypingStop` path is kept as an optimization for the common case (still removes it instantly when the stop event does arrive) rather than always waiting out the TTL, but it is not load-bearing for correctness. The `tickHandle` signal is a deliberate trick to make a `computed()` re-evaluate on a timer even though its "real" inputs (the `typingUsers` map) haven't changed — necessary because signals don't have a built-in notion of "time passing," only "value changed," so time-based expiry needs an explicit periodic nudge.

Interviewer intent: Tests whether the candidate defaults to TTL/heartbeat-based expiry (self-healing) for any "X is happening" real-time UI state, rather than a fragile explicit start/stop pairing that has no recovery path if one message is lost.

Scenario 20: Debugging duplicate real-time messages causing double-counted UI state

Situation: A real-time "cart item count" badge occasionally shows a count one higher than the actual number of items after a flaky mobile connection reconnects. Investigation shows the same `item-added` event was delivered twice — once before a brief disconnect (client didn't ack in time) and once after reconnect (server's at-least-once retry policy resent it).

Question: How do you make real-time event handlers safe against duplicate delivery, and where should that safety live architecturally?

Answer: At-least-once delivery is a very common and often unavoidable guarantee for real-time transports with retry/reconnect logic (better than the alternative of at-most-once, which would silently drop messages) — so client-side handlers for any state-mutating event must be idempotent, generally by deduping on a unique event ID rather than assuming "one event = one state change." This belongs in the store's event-application layer, not scattered across every UI component that happens to display a derived count.

```
import { Injectable, signal, computed } from '@angular/core';

interface CartItemEvent { eventId: string; type: 'item-added' | 'item-removed'; itemId: string; }

@Injectable({ providedIn: 'root' })
export class CartStore {
  private items = signal<Set<string>>(new Set());
  private processedEventIds = new Set<string>();
  private readonly maxTrackedEvents = 1000; // bound memory for the dedupe set
  private eventIdOrder: string[] = [];

  readonly itemCount = computed(() => this.items().size);

  applyEvent(evt: CartItemEvent): void {
    if (this.processedEventIds.has(evt.eventId)) {
      return; // duplicate delivery – already applied, skip entirely
    }
    this.recordProcessed(evt.eventId);

    this.items.update(set => {
      const next = new Set(set);
      if (evt.type === 'item-added') next.add(evt.itemId);
      if (evt.type === 'item-removed') next.delete(evt.itemId);
      return next;
    });
  }

  private recordProcessed(eventId: string): void {
    this.processedEventIds.add(eventId);
    this.eventIdOrder.push(eventId);
    if (this.eventIdOrder.length > this.maxTrackedEvents) {
      const oldest = this.eventIdOrder.shift()!;
      this.processedEventIds.delete(oldest);
    }
  }
}
```

Deeper reasoning: using a `Set<string>` of item IDs rather than an incrementing counter is itself a form of idempotency — adding the same item ID twice to a `Set` is a no-op, so even without explicit `eventId` dedup, a naive "count++" bug is structurally impossible here (this mirrors Scenario 8's "derive count from a set, never increment directly" lesson). The explicit `processedEventIds` dedup guards the case where the *same logical event* could otherwise be misinterpreted as two separate valid operations (relevant if, say, `item-added` events were quantity deltas rather than idempotent set-membership — always prefer designing the event payload itself to be idempotent-by-construction, like "set quantity to N" or "add item X" rather than "increment quantity

by 1", since the latter is fundamentally unsafe under at-least-once delivery no matter how much dedup you bolt on). Bounding `processedEventIds / eventIdorder` to a rolling window prevents unbounded memory growth over a long-lived session while still covering the realistic duplicate-delivery window (a duplicate arriving minutes apart due to reconnect retry is the common case; one arriving after 1000+ other events have passed is not a realistic retry window).

Interviewer intent: Tests whether the candidate designs for at-least-once delivery semantics as a default assumption for real-time transports, and whether they understand idempotent event design (set-membership, absolute-value events) as a stronger guarantee than after-the-fact deduplication alone.

Quick Revision Cheat Sheet

- **Coalesce, don't render every message.** Buffer high-frequency updates and flush once per `requestAnimationFrame` (or a fixed interval) instead of calling `.set()` on every wire message — this is the single biggest lever for taming firehose-scale real-time feeds.
- **Granular signals beat one big array signal.** Per-entity/per-row signals (`Map<id, signal>`) let only the changed row re-render; one coarse array signal forces a full re-diff on every tick.
- **Never trust arrival order — trust a logical clock.** Reject updates whose server timestamp/sequence number is not strictly newer than what's already stored; this single guard fixes out-of-order delivery, duplicate delivery, and late-arriving stale data all at once.
- **Optimistic updates should be patches layered on confirmed state via `computed()`, not snapshot/restore.** This makes concurrent, overlapping optimistic operations resolve independently instead of one rejection accidentally undoing an unrelated pending change.
- **Reconnection needs exponential backoff + jitter + a max-retry ceiling, and displayed connection state should be debounced separately from raw state** so brief blips don't spam the user with toasts.
- **Separate normalized entity state from positional/structural state (pagination, ordering).** Live field updates can apply silently; structural inserts/reorders into an already-paginated list should usually be an explicit user-triggered action ("N new items — click to load").
- **Presence (cursors, "who's online") and content sync (document edits) are different problems.** Presence is ephemeral/last-write-wins and can be throttled; concurrent content edits need CRDT/OT-grade conflict resolution, not last-write-wins.
- **`effect()` is for syncing signals to the outside world, not for feedback loops.** Guard against re-sending unchanged values, and never let a server echo feed back into the same signal that triggers the send — that's the classic infinite-loop/NG0600 trap.
- **Multi-tab duplicate connections are solved with `Sharedworker`** (single socket, fanned out via `MessagePort`) with a `BroadcastChannel`-based leader-election fallback where `Sharedworker` isn't supported.
- **Choose transport by actual requirement, not vibes.** SSE for unidirectional server push (free reconnect/resume, HTTP-friendly); WebSocket only when genuinely bidirectional/low-latency; polling only when infra truly can't support persistent connections.
- **Decouple value freshness from positional stability.** A live leaderboard's numbers can update every second while its visual ordering re-sorts on a slower, controlled cadence — plus correct `track` usage is a prerequisite for any reorder animation to be visible at all.
- **"Why did this value change" must be modeled as explicit data (source/provenance), not inferred,**

whenever the UI needs to react differently to local vs. remote-originated changes (e.g., suppressing self-triggered flash animations).

- **Shared real-time connections should be reference-counted**, torn down only when the last consumer releases it (via `DestroyRef.onDestroy`), not tied to any single component's lifecycle.
- **Authorization for persistent connections is continuous, not a handshake-time check.** Client-side filtering is UX only; the server must re-scope or stop pushing data the instant permission is revoked.
- **Ephemeral, time-bound real-time state (typing indicators, "user is online") should expire via TTL/heartbeat, not rely on a paired start/stop event** that has no recovery path if the stop message is ever dropped.
- **Assume at-least-once delivery.** Dedupe by event ID and, more fundamentally, design event payloads to be idempotent by construction (set-membership, absolute values) rather than relative deltas that break under duplicate delivery.
- **Keep transport code (socket/SSE plumbing) and store business logic (merge/ordering/dedup) separately testable.** Pure, synchronous store methods over signals can be unit-tested without `fakeAsync` / `tick()`, eliminating a common source of flaky real-time tests.

Created By - Durgesh Singh